APRENDIZAJE AUTOMÁTICO · CIENCIA DE DATOS

MLOps: del notebook a producción

Juan Pablo Jimenez Quiroz
Juan Diego Robless De La Ossa
Universidad EAFIT

Repositorio en GitHub

CONTEXTO

Un notebook no es un producto

1. ¿Qué pasa si cambian los datos?
2. ¿Quién reentrena, y cuándo?
3. ¿Cómo reproduzco el modelo de hace 3 meses?



Sculley et al., 2015

CONTEXTO

¿Qué es MLOps?

Machine Learning + Operations

En ML cambian tres cosas:

</>

Código

Datos

Modelo

Git no basta: versionar y vigilar las tres

CONTEXTO

Ciclo de vida de un modelo



CONTEXTO

Niveles de madurez

NIVEL 0

Manual

NIVEL 1

Pipeline de entrenamiento automatizado

NIVEL 2

CI/CD automatizado

Esta demo: entre el nivel 1 y el 2

COMPONENTES Y DESPLIEGUE

Versionado y experiment tracking

CÓDIGO	DATOS	MODELOS	EXPERIMENTOS
Git	DVC	Registro de modelos	MLflow

MLflow guarda: Parámetros · Métricas · Artefactos

COMPONENTES Y DESPLIEGUE

CI / CD / CT

CI

Integración continua

- Pruebas de código
- Pruebas de datos

CD

Entrega continua

- Despliegue automático
- Si las pruebas pasan

CT

Entrenamiento continuo

- Reentrenar si el modelo se degrada

En el repo: GitHub Actions + imagen Docker

COMPONENTES Y DESPLIEGUE

Estrategias de despliegue

CÓMO SE SIRVE

1. **Batch**
2. **En línea (API REST)**
   Lo que usamos en la demo
3. **Embebido**

CÓMO SE LANZA UNA VERSIÓN

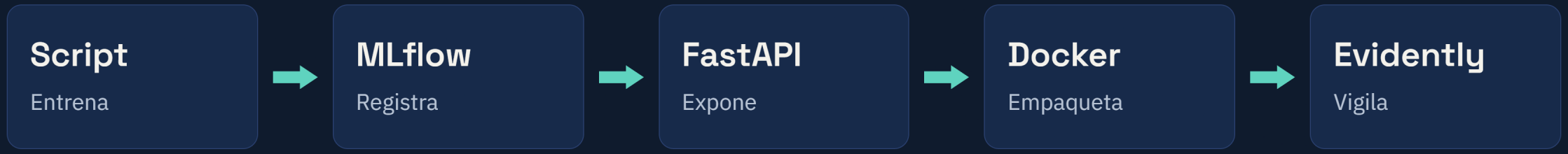
1. **Shadow**
2. **Canary**
3. **Blue-green**

Compromiso: latencia · costo · riesgo · complejidad

COMPONENTES Y DESPLIEGUE

Caso de la demo: churn

Abandono de clientes en telecomunicaciones



Modelo simple a propósito: el foco es el proceso

DEMO Y MONITOREO

Monitoreo: qué puede salir mal

Data drift

Cambian las entradas

Concept drift

Cambia la relación con la respuesta

Etiquetas tardías

Vigilar primero las entradas

Reglas de decisión

1 Umbrales 2 Alertas 3 Reentrenar

DEMO Y MONITOREO

Demo: paso a paso

ENTRENAR Y DESPLEGAR

1. make train
2. MLflow: comparar corridas
3. Promover la mejor corrida
4. Docker: build y run
5. curl a la API: probabilidad de churn

DETECTAR DRIFT

6. Datos de producción: cargos +30%
7. Reporte de Evidently
8. MonthlyCharges con drift
9. El desempeño del modelo cae

DEMO Y MONITOREO

Ante un drift: decidir

¿Cambio legítimo o error en los datos?

LEGÍTIMO

1. Reentrenar con datos recientes
2. Validar el nuevo modelo
3. Desplegar con canary

ERROR

1. Corregir el pipeline de datos
2. No tocar el modelo

CIERRE

Limitaciones y trade-offs

Feature store
No incluido

Orquestador
Airflow, Kubeflow

Escalado
Mucho tráfico

Seguridad y gobernanza
No abordadas

Drift simulado
En real es más ruidoso

Costo de complejidad
Cada herramienta se mantiene

Un nivel 1 bien hecho vale más que un nivel 2 a medias

CIERRE

Tres ideas para llevarse

1. El modelo es una pieza pequeña del sistema
2. Reproducible = código + datos + modelo
3. Desplegar es el comienzo, no el final

¿Preguntas?

Repositorio en GitHub